

## Práctica Bash

Estudiante: Limber Mamani Choque

### Sección A — Navegación y rutas

1. Abre una terminal y, sin usar el mouse, responde: ¿en qué carpeta empiezas? ¿Qué comando te lo dijo?

R. inicias en tu carpeta personal, vemos con el comando `pwd`

2. Lista el contenido de tu carpeta actual incluyendo los archivos ocultos. ¿Qué distingue a un archivo oculto en Linux?

R. Comando: `ls -a` , Lo que los distingue es que su nombre comienza con un punto (.)

3. Estás en `/home/tu_usuario`. Escribe DOS comandos distintos para llegar a `/home/tu_usuario/Documentos`: uno con ruta relativa y otro con ruta absoluta.

R. Ruta absoluta: `cd /home/tu_usuario/Documentos`

Ruta relativa (asumiendo que ya estás en `/home/tu_usuario`): `cd Documentos`

4. Escribe qué significa cada símbolo: `.` `..` `~` `/`

R.

| Símbolo         | Significado                                                                                                                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>.</code>  | Directorio actual. También se usa para ejecutar un script en el shell actual ( <code>. script.sh</code> ).                                                                                                           |
| <code>..</code> | Directorio padre (un nivel arriba del directorio actual).                                                                                                                                                            |
| <code>~</code>  | Directorio personal (home) del usuario actual.                                                                                                                                                                       |
| <code>/</code>  | Directorio raíz (root) del sistema de archivos. Es el nivel más alto de la jerarquía de directorios. También se usa como separador de directorios en las rutas, por ejemplo: <code>/home/usuario/documentos</code> . |

5. Estando en `/etc`, vuelve a tu carpeta personal con UN solo comando (hay al menos dos formas: anota ambas).

R.

|                 |                   |
|-----------------|-------------------|
| <code>cd</code> | <code>cd ~</code> |
|-----------------|-------------------|

6. ¿Para qué sirve la carpeta `/home`? ¿Y `/etc`? ¿Y `/tmp`? (Una frase cada una.)

R.

|                    |                                                                        |
|--------------------|------------------------------------------------------------------------|
| <code>/home</code> | Contiene las carpetas personales de los usuarios y sus archivos.       |
| <code>/etc</code>  | Almacena los archivos de configuración del sistema y de los programas. |
| <code>/tmp</code>  | Guarda archivos temporales que utilizan el sistema y las aplicaciones. |

### Sección B — Ficheros y directorios

7. Crea, con la MENOR cantidad de comandos posible, esta estructura:

```
examen/
├── practica/
│   ├── notas.txt
│   └── borrador.txt
└── respaldo/
```

R. `mkdir -p examen/practica/respaldo && touch examen/practica/notas.txt`  
`examen/practica/borrador.txt`

8. Copia `notas.txt` dentro de `respaldo/` y luego renombra el original a `apuntes.txt`. ¿Qué comando usaste para renombrar y por qué no existe un comando “rename” clásico?

R.

o `cp examen/practica/notas.txt examen/practica/respaldo/`

- `mv examen/practica/notas.txt examen/practica/apuntes.txt`
- En Linux, el comando `mv` (move) se usa tanto para mover como para renombrar.

9. ¿Cuál es la diferencia entre `rm`, `rmdir` y `rm -r`? ¿Cuál usarías para borrar la carpeta respaldo con su contenido?

R.

|                    |                                                    |
|--------------------|----------------------------------------------------|
| <code>rm</code>    | Borra archivos.                                    |
| <code>rmdir</code> | Borra directorios <b>solo si están vacíos</b> .    |
| <code>rm -r</code> | Borra directorios y todo su contenido (recursivo). |

Para borrar `respaldo` con todo dentro: `rm -rf respaldo/`

10. Con UN solo comando, copia todos los archivos `.txt` de `practica/` hacia `respaldo/`.

R. `cp practica/*.txt respaldo/`

11. Verdadero o falso (justifica): “Si borro un archivo con `rm`, puedo recuperarlo de la papelera”.

R. **Falso**. Cuando usas `rm` en la terminal, el archivo se desvincula inmediatamente del sistema de archivos; no pasa por la papelera de escritorio (GUI).

### Sección C — Redirección, tuberías y permisos

12. ¿Qué diferencia hay entre `>` y `>>`? Escribe un ejemplo donde usar el equivocado te haga perder información.

R.

|                       |                                                                       |
|-----------------------|-----------------------------------------------------------------------|
| <code>&gt;</code>     | Sobrescribe el archivo (si existía, se pierde el contenido anterior). |
| <code>&gt;&gt;</code> | Añade (append) al final, manteniendo el contenido previo.             |

13. Guarda en `inventario.txt` el listado largo de tu carpeta personal, y luego añádele la fecha actual sin borrar el listado.

R.

- `ls -l > inventario.txt`
- `date >> inventario.txt`

14. Explica paso a paso qué hace: `ls -l /etc | grep conf | wc -l`

R.

1. `ls -l /etc`: Lista en formato largo el contenido de `/etc`.
2. `|` (pipe): Pasa esa lista al siguiente comando.
3. `grep conf`: Filtra solo las líneas que contienen la palabra "conf".
4. `wc -l`: Cuenta cuántas líneas resultaron del filtro. Resultado: La cantidad de archivos/carpetas en `/etc` que contienen "conf" en su nombre.

15. Descifra estos permisos numéricos a letras: 755 640 700 644

R.

- 755: `rw-r-xr-x`
- 640: `rw-r-----`
- 700: `rw-x-----`
- 644: `rw-r--r--`

16. Traduce a números: `rw-r-x--x` y `rw-rw-r--`

R.

- `rw-r-x--x`: 751
- `rw-rw-r--`: 664

17. ¿Por qué los valores de los permisos son 4, 2 y 1 y no, por ejemplo, 3, 2 y 1? (Pista: unicidad de las sumas.)

R. por el sistema binario: lectura =  $2^2=4$ , escritura =  $2^1=2$ , ejecución =  $2^0=1$ . Al usar potencias de 2, cada combinación de permisos suma un valor único, evitando ambigüedades

18. Un compañero ejecuta `./script.sh` y recibe “Permiso denegado”. ¿Qué pasó y cómo se arregla?

R. El archivo no tiene el bit de ejecución activo. Se arregla con: `chmod +x script.sh`

## Sección D — Scripts y condicionales

19. ¿Qué hace la línea `#!/bin/bash` y dónde debe ir?

R. La línea: `#!/bin/bash` se llama shebang.

- Indica que el script debe ejecutarse con el intérprete Bash, ubicado en `/bin/bash`.
- Debe ir siempre en la primera línea del script.

20. Encuentra los DOS errores de este script y corrígelo:

|                                                                                         |                                                                                                               |
|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <pre>#!/bin/bash nombre = "Ana" if [\$nombre = "Ana"] then     echo "Hola Ana" fi</pre> | <p>Corregido:</p> <pre>#!/bin/bash  nombre="Ana"  if [ "\$nombre" = "Ana" ] then     echo "Hola Ana" fi</pre> |
|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|

21. Escribe un script `temperatura.sh` que pida una temperatura y responda: 30 o más → “Calor”; entre 15 y 29 → “Templado”; menos de 15 → “Frío”.

R.

```
#!/bin/bash

read -p "Ingrese la temperatura: " temp

if [ "$temp" -ge 30 ]
then
    echo "Calor"
elif [ "$temp" -ge 15 ]
then
    echo "Templado"
else
    echo "Frío"
fi
```

22. ¿Qué imprime este script si el usuario escribe 25? ¿Y si escribe 90?

|                                                                                                                                                            |                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>#!/bin/bash  read -p "Número: " n  if [ \$n -ge 50 ] then     echo "Grande" elif [ \$n -ge 20 ] then     echo "Mediano" else     echo "Pequeño"</pre> | <p>25 Imprime: Mediano<br/>Porque:<br/>¿<math>25 \geq 50</math>? → No.<br/>¿<math>25 \geq 20</math>? → Sí.</p> <p>90 Imprime: Grande<br/>Porque:<br/>¿<math>90 \geq 50</math>? → Sí.</p> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

23. Sin ejecutarlo: ¿qué imprime `echo "Hola $USER, estás en $PWD"`? ¿De dónde salen esos valores si nadie los declaró en el script?

R. Hola limber, estás en /home/limber

USER → nombre del usuario actual.

PWD → directorio de trabajo actual.

No es necesario declararlas dentro del script porque ya existen en el entorno del sistema.

## Sección E — Bucles

24. Escribe un `for` que imprima los números del 1 al 5 usando un rango.

R.

```
#!/bin/bash
for i in {1..5}
do
    echo $i
done
```

25. Escribe un `while` que haga exactamente lo mismo que el ejercicio anterior. ¿Cuál preferirías aquí y por qué?

R.

```
#!/bin/bash
i=1
while [ $i -le 5 ]
do
    echo $i
    i=$((i + 1))
done
```

Preferiría el `for`, porque ya se conoce el rango de números (del 1 al 5). Es más corto, claro y fácil de leer.

26. ¿Qué hace este script? ¿Cuántas líneas imprime si la carpeta tiene 3 archivos `.html` y 2 `.css`?

```
#!/bin/bash
for f in *.html
do
    echo "Web: $f"
done
```

Recorre todos los archivos cuyo nombre termine en `.html` y muestra un mensaje con el nombre de cada uno

Por ejemplo, si existen:

```
index.html
contacto.html
nosotros.html
```

La salida será:

```
Web: index.html
Web: contacto.html
Web: nosotros.html
```

¿Cuántas líneas imprime?

```
3 archivos .html
2 archivos .css
```

Imprime 3 líneas, porque el bucle solo procesa los archivos `.html`.

R.

27. Este script tiene un problema grave. ¿Cuál es, qué pasará al ejecutarlo, y cómo se detiene y se corrige?

|                                                                                                    |                                                                                                                            |
|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <pre>#!/bin/bash i=1 while [ \$i -le 10 ] do echo "Vuelta \$i" done Desafío final integrador</pre> | <p>La variable <code>i</code> <b>nunca cambia de valor</b>, por lo tanto el script entrará en un <b>bucle infinito</b></p> |
|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|

### Desafío final integrador

28. Escribe `organizador.sh`, un script que: (a) cree la carpeta `revision` si quieres apoyarte en ella; (b) recorra todos los elementos de la carpeta actual con un bucle; (c) para cada uno, imprima “[CARPETA] nombre” si es un directorio o “[FICHERO] nombre” si no; y (d) al final imprima cuántos elementos revisó en total.  
Pistas: `[ -d ]`, un contador con `$(())`.

|                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>#!/bin/bash  contador=0  for elemento in * do     if [ -d "\$elemento" ]     then         echo "[CARPETA] \$elemento"     else         echo "[FICHERO] \$elemento"     fi      contador=\$((contador + 1)) done  echo "Total de elementos revisados: \$contador"</pre> | <p><code>for elemento in *</code> → Recorre todos los elementos de la carpeta actual.</p> <p><code>if [ -d "\$elemento" ]</code> → Verifica si el elemento es un directorio.</p> <p>Si lo es, imprime <code>[CARPETA] nombre</code>.</p> <p>Si no, imprime <code>[FICHERO] nombre</code>.</p> <p><code>contador=\$((contador + 1))</code> → Incrementa el contador en cada iteración.</p> <p>Al finalizar, muestra el número total de elementos revisados.</p> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|